

Fo λ ang in Depth

Concepts, Constructs,
and Systems



Kameswara Rao Mithipati

Preface

Why This Book Exists

A programming language is more than syntax; it is the lens through which we view a system's architecture. It shapes how we define boundaries, express state, and whether complexity feels manageable or overwhelming.

FoLang was designed with that in mind.

The FoLang Language Reference answers the formal question:

What is FoLang?

It defines the syntax, semantics, type rules, source structure, metadata rules, capability boundaries, and object model. It is the normative specification, and where exact behaviour matters, it governs.

This book answers a different question:

Why is each construct shaped the way it is, and when should you reach for it?

A specification tells you that a construct exists and how it behaves. It does not tell you what problem the construct was built to solve, what it costs, which alternative to consider first, or what happens when you choose wrongly. That is the space this book occupies.

What This Book Covers

Every construct in the language, treated in depth.

For each one, the same questions:

- **What problem does it solve?** Constructs exist for reasons. Knowing the reason usually settles when to use it.
- **How does it behave?** The precise semantics, with the reference cited where the exact rule matters.

- **What does it cost?** Every abstraction has a price — in verbosity, in coupling, in what it forbids elsewhere.
- **What are the alternatives?** Most problems admit several FoLang solutions. The interesting question is which one fits.
- **When is it the wrong choice?** Often the most useful section.

Where FoLang offers several constructs of similar nature, the book develops the reasoning once and then shows how it transfers. The goal is that you can reason about a construct the book spends less time on, because the reasoning generalises.

What This Book Assumes

You have written programs in at least one other language and are comfortable with types, functions, and objects.

You do not need to have used FoLang. You do not need a particular background language. Where FoLang differs from what you may be used to, the book says so directly — but it explains FoLang on its own terms rather than as a set of translations from somewhere else.

That distinction matters more than it sounds. Similar syntax does not imply similar semantics, and a construct understood as “the FoLang version of X” tends to be used the way X would be used, which is often not the way it works best.

The FoLang Model

Before the constructs, the model they live in.

FoLang starts from a uniform premise:

Everything in FoLang is an object.

Managed objects use reference semantics by default, so assigning one copies a reference rather than duplicating contents. Value equality and reference identity remain separate questions. Mutation, immutability, sharing, copy-on-write behaviour, generic abstraction, matching, functions, and user-defined types all live within a single model rather than several unrelated ones.

There are deliberate exceptions where a problem domain calls for them.

`co.lang.cstruct` is an ABI-oriented value-semantic representation, because foreign function interfaces require one. FoLang does not force every feature into a single implementation strategy simply to keep the surface uniform.

What stays constant is the reasoning. Once you know how FoLang thinks about objects, identity, and boundaries, most constructs become predictable rather than memorised.

A Small Example

Consider two employee records holding the same values:

```
a Employee = Employee{name: "Rao", id: 1};
b Employee = Employee{name: "Rao", id: 1};

a == b;           // true  - same values
a.sameRef(b);     // false - different objects
```

Think of a shelf holding ten identical jars of coffee beans. Are they equal? Yes — same beans, same weight, same price. Are they the same jar? No. You take one and I take another. Equal, but not the same.

Now bind a third name:

```
c := a;

c == a;           // true
c.sameRef(a);     // true  - c and a are the same object
```

You have handed me your jar. We are each holding a jar, and it is the same jar. If I pour some out, yours has less too, because there was only ever one.

`==` asks the first question. `sameRef` asks the second. `:=` hands over the jar.

This distinction is not a language quirk. It is how objects behave, and FoLang keeps the two questions apart rather than letting one silently answer the other. The analogy holds for managed objects; value-semantic representations such as `co.lang.cstruct` behave differently, and the book covers where that line falls.

How This Book Is Organised

Part I — The Model establishes what everything else rests on: objects, bindings, references, equality, mutation, and the boundaries between them.

Part II — Structure covers the constructs that shape a program: structs and companion units, classes, units, modules, and the composition forms.

Part III — Types and Abstraction covers types, generics, interfaces, signatures, type classes and instances, refinement types, and pattern matching.

Part IV — Programs and Systems moves from declarations to systems: packages, project layout, components, libraries and exports, metadata, capability boundaries, and execution models.

Part V — Advanced Facilities covers mechanisms for cases the ordinary abstractions do not reach: dependent types, macros, operators, reflection, specialisation, and native interfaces.

Each part builds on the last, but the construct chapters are written to be read individually once the model in Part I is familiar.

Reading the Examples

FoLang uses explicit statement termination. A simple statement ends with `;`, while a block or declaration body ends with its closing `}`.

```
name co.lang.string = "Rao";
age  co.lang.int = 30;
```

```
(co.const.true).then({
  co.out.println("ready");
}).default({
  co.out.println("not ready");
});
```

Examples favour complete, readable forms first. Shorter forms appear once their meaning is established.

Where an example shows something the language rejects, it is marked. Those are often more instructive than the working cases, because a rule is easiest to understand at its boundary.

A Note on Stability

FoLang is approaching version 1.0 but has not reached it. During the alpha period, syntax and semantics may be refined, and examples in this book are updated to track the specification.

Where this book and the language reference disagree, the reference is correct. Report the discrepancy — it means one of the two needs fixing.

Before We Begin

The first chapter does not start with a catalogue of keywords.

It starts with the model, because nearly every design decision in FoLang follows from it:

How does FoLang think about a program?

Once that is in place, the constructs stop being a list to memorise and start being a set of tools with obvious applications.